ARQUITECTURA DE COMPUTADORES. 2º GII-IS
Departamento de Arquitectura y Tecnología de Computadores
Universidad de Sevilla

Boletín para las sesiones de laboratorio

COMPUTACIÓN PARALELA CON COMPUTADORES DE MEMORIA DISTRIBUIDA USANDO MPI

PREPARACIÓN

INTRODUCCIÓN Y OBJETIVOS

El Problema: Un solo ordenador ya no es suficiente

En la práctica anterior vimos que "la barra libre se había acabado" y aprendimos a usar OpenMP para exprimir al máximo los múltiples núcleos de un procesador compartiendo la misma memoria. Sin embargo, ¿qué ocurre cuando nuestro problema es tan gigantesco que ni siquiera el procesador más potente del mundo tiene suficientes núcleos para resolverlo en un tiempo razonable, o cuando la memoria RAM necesaria excede a la disponible en una única máquina? Nos topamos con un nuevo límite físico y económico: escalar un único ordenador gigante (supercomputador de memoria compartida) es carísimo y tiene limitaciones técnicas.

La solución del mundo real (utilizada por Google, la NASA o los centros de investigación) es el clúster: conectar decenas, cientos o miles de ordenadores normales a través de una red para que trabajen juntos en un fin común. En términos generales, esta solución ofrece un alto rendimiento a un coste asequible.

Esto nos lleva a un nuevo desafío: en un clúster, la memoria está distribuida. El ordenador A no puede leer directamente las variables que están en la memoria RAM del ordenador B. Si queremos que colaboren, tenemos que cambiar de paradigma y enseñarles a "hablar" entre ellos.

La Solución: Memoria Distribuida y MPI

Para programar en estas arquitecturas, necesitamos abandonar la idea de los "hilos" compartiendo variables y adoptar un modelo donde procesos completamente independientes se comunican explícitamente enviándose datos a través de la red.

Aquí es donde entra **MPI (Message-Passing Interface)**.

MPI no es un lenguaje nuevo, sino el paradigma más empleado hoy en día en plataformas de memoria distribuida. En lugar de directivas de compilación como en OpenMP, MPI es una biblioteca de funciones que permite a los procesos intercambiar información de forma estandarizada.

Hay un concepto clave que marca la diferencia en este modelo: cuando ejecutamos un programa con MPI, lo que ocurre realmente es que se lanzan simultáneamente múltiples procesos independientes y todos ellos ejecutan exactamente el mismo archivo de código fuente.

¿Cómo es posible entonces que hagan cosas distintas? La respuesta está en que cada proceso puede obtener su propio identificador único o "rango". Gracias a esta identidad, podemos utilizar simples sentencias condicionales en nuestro código. Así conseguimos que, aunque todos los procesos ejecuten el mismo código, cada uno tome un camino diferente, se encargue de una parte distinta del cálculo local y se comuniquen entre sí para construir la solución global.

En esta práctica, utilizaremos MPI junto con el lenguaje de programación Python (mpi4py). Aprenderéis a orquestar decenas de procesos usando dos mecanismos fundamentales:

- **Comunicaciones punto a punto:** Mensajes directos entre un emisor y un receptor (usando send y recv).

- **Comunicaciones colectivas:** Operaciones globales optimizadas (como scatter, gather, reduce o bcast) que permiten repartir o recolectar enormes bloques de datos (como vectores o matrices) entre todos los procesos del clúster con una sola instrucción.

Objetivos de la práctica

El objetivo fundamental es que conozcas la metodología para desarrollar aplicaciones paralelas para computadores de memoria distribuida y te familiarices con sus particularidades de ejecución, depuración y rendimiento.

Al finalizar esta práctica, serás capaz de:

1. **Desarrollar** programas paralelos en Python usando la biblioteca de funciones MPI para sistemas Linux.
2. **Aprovechar** los múltiples núcleos de un procesador simulando un entorno distribuido en una única máquina.
3. **Desplegar** y lanzar la ejecución de una aplicación paralela en varios PCs reales, configurándolos mediante SSH para que trabajen como distintos nodos de un clúster.
4. **Analizar** el rendimiento de tus soluciones midiendo tiempos de ejecución y calculando matemáticamente la aceleración obtenida.

Conceptos Fundamentales de MPI con Python (mpi4py)

Para desarrollar las aplicaciones de esta práctica, emplearemos el lenguaje Python junto con la biblioteca mpi4py. A continuación, se detallan los mecanismos y funciones clave que estructuran un programa bajo el paradigma MPI:

A. Ejecución del código

A diferencia de un programa secuencial estándar, el código MPI se lanza especificando desde la línea de comandos cuántas instancias simultáneas queremos ejecutar. Debes usar el comando mpirun indicando el número de procesos simultáneos que deseas lanzar mediante el parámetro -np X.

- **Comando básico:** `mpirun -np X python3 nombre_archivo.py`

B. Identificación: El "Mundo" MPI

Dado que todos los procesos ejecutan simultáneamente el mismo código fuente, necesitan una forma de orientarse y determinar qué parte del trabajo les corresponde.

- **MPI.COMM_WORLD:** Es el comunicador global por defecto, el cual agrupa a todos los procesos que han sido lanzados en la ejecución.
- **comm.Get_rank():** Devuelve el identificador único (o rango) del proceso actual dentro del comunicador (desde 0 hasta X-1, donde X es el número de procesos).
- **comm.Get_size():** Devuelve el número total de procesos que se están ejecutando.
- **MPI.Get_processor_name():** Permite conocer el nombre del nodo de red en el que se está ejecutando un proceso específico (muy útil cuando pasemos a usar varios PCs).

C. Comunicaciones Punto a Punto

Constituyen el mecanismo más básico de intercambio de datos, donde la información viaja directamente entre un proceso emisor y un proceso receptor (por ejemplo, del proceso 1 al proceso 0).

- **send(datos, dest):** Envía un mensaje al proceso destino.
- **recv(source):** Recibe un mensaje del proceso origen.

Nota importante: Aunque la teoría dice que send es bloqueante, en la versión de MPI de nuestros laboratorios solo bloquea si el mensaje es mayor de 4 KiBytes. Si el mensaje es más pequeño, MPI lo guarda en un buffer interno y el programa continúa su ejecución inmediatamente.

D. Comunicaciones Colectivas

Diseñar envíos punto a punto para distribuir estructuras de datos masivas resulta ineficiente. Las operaciones colectivas resuelven esto involucrando a todos los procesos del comunicador de forma altamente optimizada. En mpi4py existen dos formas de invocar estas operaciones. Las funciones en **minúsculas** (como gather) gestionan objetos genéricos de Python y devuelven el resultado como un valor de retorno. Las funciones en **MAYÚSCULAS** (como Gather) están optimizadas para **arrays de NumPy**, son mucho más rápidas y requieren que les pasemos el buffer donde se guardará el resultado.

- **dato_local = comm.scatter(lista, root) / comm.Scatter(lista, dato_local, root):** Sirve para repartir datos desde un proceso raíz a todos los demás. La versión en minúscula **devuelve** el trozo, mientras que la versión en mayúscula lo escribe en la variable dato_local que se pasa como parámetro.
- **dato = comm.bcast(dato, root) / comm.Bcast(datos, root)**: El proceso raíz envía una copia completa e idéntica de la estructura de datos a todos los procesos del comunicador. La versión en minúscula **devuelve** el dato, mientras que la versión en mayúscula lo escribe
- **lista_completa = comm.gather(dato_local, root) / comm.Gather(dato_local, lista_completa, root)**: Realiza la operación inversa a Scatter. Reúne los datos parciales de cada proceso y los pega en orden en una estructura final dentro del proceso raíz. La versión en minúscula **devuelve** la lista completa, mientras que la versión mayúscula lo escribe en la variable lista_completa que se pasa como parámetro.
- **resultado = comm.reduce(datos, op, root) / comm.Reduce(datos, resultado, op, root)**: Recolecta datos de todos los procesos en un comunicador, los combina mediante una operación matemática o lógica (como MPI.SUM, MPI.MAX o MPI.MIN) y entrega el resultado final a un único proceso específico (típicamente el 0 o root). La versión en minúscula **devuelve** el resultado, mientras que la versión mayúscula lo escribe en la variable resultado que se pasa como parámetro.
- **comm.Allreduce(datos, resultado, op)**: Opera de manera similar a reduce, pero con la diferencia fundamental de que el resultado final de la reducción se envía a todos los procesos en lugar de solo al maestro. Es una operación de "muchos a muchos". Equivale a realizar un Reduce seguido de un Bcast. Todos los procesos aportan su dato local y, al terminar la operación, **todos los procesos tienen el resultado final** en su parámetro de salida, que en este caso es la variable resultado que se pasa como parámetro.

Operaciones soportadas en reducciones (op):

- **MPI.SUM**: Suma de todos los valores.
- **MPI.MAX**: Valor máximo.
- **MPI.MIN**: Valor mínimo.
- **MPI.PROD**: Producto de los valores.
- **MPI.LAND**: Operación lógica AND.
- **MPI.LOR**: Operación lógica OR.

A continuación se muestran ejemplos de uso de las funciones anteriores, en los que suponemos que se lanzan 4 procesos:

1. Difusión (*Broadcast*)

dato = comm.bcast(dato, root)

El proceso *root* envía el *dato* (que es un objeto Python) a todos los procesos. La función devuelve el dato pasado como parámetro por el proceso *root*. En el caso del proceso *root* este recibe el mismo valor que pasó como parámetro.

Por ejemplo, al ejecutar *dato = comm.bcast(dato, root=0)*:

El Proceso 0 (el emisor) mantiene el valor la variable *dato* (guarda en *dato* el mismo valor que él pasó como parámetro).

El Proceso 1 recibe una copia del dato enviado por el proceso 0 y lo guarda en su propia variable *dato*. La función ignora el valor del parámetro *dato* del proceso 1.

El Proceso 2 recibe una copia del dato enviado por el proceso 0 y lo guarda en su propia variable *dato*. La función ignora el valor del parámetro *dato* del proceso 2.

El Proceso 3 recibe una copia del dato enviado por el proceso 0 y lo guarda en su propia variable *dato*. La función ignora el valor del parámetro *dato* del proceso 3.

comm.Bcast(buffer, root)

El proceso *root* transmite el contenido de un *array* (ej. NumPy) a los *buffers* de los demás.

2. Distribución (*Scatter*)

trozo = comm.*scatter*(lista, root):

El *root* reparte una lista de objetos. Cada proceso recibe su parte como valor de retorno. Por ejemplo: supongamos que el proceso *root* tiene la variable lista = [A, B, C, D]. Al ejecutar *trozo = comm.scatter(lista, root=0)*:

El Proceso 0 recibe A en su variable *trozo*.

El Proceso 1 recibe B en su variable *trozo*.

El Proceso 2 recibe C en su variable *trozo*.

El Proceso 3 recibe D en su variable *trozo*.

comm.Scatter(sendbuf, recvbuf, root):

Distribuye un *array* de NumPy. Divide el *sendbuf* del *root* y llena el *recvbuf* de cada proceso.

3. Recolección (*Gather*)

lista = comm.gather(dato, root):

Los procesos envían un objeto al *root*, quien los recibe todos en una lista ordenada por *rank*. La lista se obtiene como valor de retorno de la función.

Por ejemplo, supongamos los siguientes valores en la variable *dato*:

El Proceso 0 tiene dato = A

El Proceso 1 tiene dato = B

El Proceso 2 tiene dato = C

El Proceso 3 tiene dato = D

Al ejecutar l*ista = comm.gather(dato, root=0)*:

El Proceso 0 (*root*) recibe la lista completa [A, B, C, D] en su variable lista.

El Proceso 1 recibe *None* en su variable lista.

El Proceso 2 recibe *None* en su variable lista.

El Proceso 3 recibe *None* en su variable lista.

comm.Gather(sendbuf, recvbuf, root):

La versión para *arrays*. Los datos de *sendbuf* de cada proceso se concatenan en el *recvbuf* del *root*.

4. Reducción (*Reduce*)

res = comm.reduce(dato, op, root):

Combina objetos (ej. enteros o *floats*) usando la operación *op*. El resultado final se devuelve solo en el proceso *root*. Ejemplo: supongamos el siguiente estado inicial:

El Proceso 0 tiene dato = 10

El Proceso 1 tiene dato = 20

El Proceso 2 tiene dato = 30

El Proceso 3 tiene dato = 40

Al ejecutar *res = comm.reduce(dato, op=MPI.SUM, root=0)*:

El Proceso 0 (*root*) recibe en su variable *res* el resultado de la suma: 100.

El Proceso 1 recibe *None* en su variable res.

El Proceso 2 recibe *None* en su variable res.

El Proceso 3 recibe *None* en su variable res.

comm.Reduce(sendbuf, recvbuf, op, root):

Realiza la operación matemática sobre los elementos de los *arrays sendbuf* y guarda el resultado en el *array recvbuf* del *root*.

5. Reducción Total (*Allreduce*)

res = comm.allreduce(dato, op):

Igual que reduce, pero todos los procesos obtienen el valor de retorno con el resultado. No hay parámetro *root*.

Ejemplo: supongamos el siguiente estado inicial:

El Proceso 0 tiene dato = 10

El Proceso 1 tiene dato = 20

El Proceso 2 tiene dato = 30

El Proceso 3 tiene dato = 40

Al ejecutar *res = comm.allreduce(dato, op=MPI.SUM*):

El Proceso 0 recibe el resultado de la suma: 100 en su variable res.

El Proceso 1 recibe el resultado de la suma: 100 en su variable res.

El Proceso 2 recibe el resultado de la suma: 100 en su variable res.

El Proceso 3 recibe el resultado de la suma: 100 en su variable res.

comm.Allreduce(sendbuf, recvbuf, op):

Versión de alto rendimiento para *arrays*. Todos los procesos terminan con el resultado de la operación en su *recvbuf*.

Tal y como se ha indicado hay que tener en cuenta un detalle con los nombre de las funciones en MPI y Python (mpi4py). Existen dos familias de métodos para las mismas operaciones:

- Métodos con minúsculas (*gather, scatter, bcast*): Trabajan con objetos genéricos de Python (listas, diccionarios). Usan *pickle*[1] para serializar los datos, lo que los hace flexibles pero lentos. El resultado se recibe como un valor de retorno.

- Métodos con mayúsculas (*Gather, Scatter, Bcast*): Están optimizados para objetos que soportan el protocolo de *buffer*, típicamente *arrays* de NumPy. Son muchísimo más rápidos porque se comunican directamente en C. Los datos se reciben en un parámetro de salida que debe estar pre-asignado.

---

[1]Nota técnica: *Pickle* es el protocolo estándar de Python para la "serialización" de objetos. Convierte cualquier estructura de datos (listas, diccionarios, objetos propios) en un flujo de bytes para poder enviarlos a través de la red (MPI). Aunque es muy flexible, requiere un esfuerzo extra de CPU para empaquetar y desempaquetar, por lo que es menos eficiente que el paso de *buffers* directos (*NumPy*).

Medición del rendimiento

En computación paralela, no basta con que el programa funcione correctamente; el objetivo principal es que funcione más rápido. Para cuantificar esta mejora, necesitamos medir el tiempo de ejecución con precisión.

Para medir el tiempo en un entorno distribuido, utilizaremos la función estándar **MPI.Wtime(),** que devuelve el tiempo actual en segundos. Al igual que en la práctica anterior, no mediremos el tiempo de CPU sino el tiempo de reloj (véase el Tema 2).

Para obtener una medición fiable en un clúster, es vital asegurar que todos los procesos comiencen a contar al mismo tiempo. Para ello, usamos **comm.Barrier()** justo antes de la toma de tiempos.

```
# 1. Sincronizamos a todos los procesos
comm.Barrier()

# 2. El proceso 0 (o todos) captura el tiempo de inicio
t_inicio = MPI.Wtime()

# --- CÓDIGO PARALELO A MEDIR ---
# (Por ejemplo, el reparto de datos y el cálculo)
# -------------------------------

# 3. Sincronizamos de nuevo antes de parar el cronómetro
comm.Barrier()
t_fin = MPI.Wtime()

if rank == 0:
    tiempo_total = t_fin - t_inicio
    print(f"Tiempo paralelo: {tiempo_total:.6f} segundos")
```

La métrica fundamental para evaluar tu práctica es el **factor de aceleración** (véase el Tema 2). Este valor nos indica cuántas veces más rápido es tu programa paralelo (al usar múltiples procesos) en comparación con la ejecución secuencial (con un solo proceso).

$$Aceleración = \frac{Rendimiento_{par}}{Rendimiento_{sec}} = \frac{T_{sec}}{T_{par}}$$

Ejemplos de interpretación:

- **Aceleración = 2:** El programa paralelo es el doble de rápido**.**
- **Aceleración = 4:** El programa paralelo es cuatro veces más rápido**.**
- **Aceleración < 1: ¡Cuidado!** Tu versión paralela es más lenta que la secuencial (probablemente por el coste de crear los hilos o por demasiada sincronización).

Ejemplo:

Si el programa original tardaba 10 segundos y tu versión con 4 hilos tarda 2.5 segundos:

$$Aceleración = \frac{T_{sec}}{T_{par}} = \frac{10}{2.5} = 4$$

¡Has conseguido una aceleración perfecta o lineal!

METODOLOGÍA

Se estudiarán los fundamentos de la programación utilizando el paradigma más empleado hoy en día en este tipo de plataforma: la biblioteca de funciones MPI (Message-Passing Interface), junto con el lenguaje de programación Python.

Se realizarán programas para un PC con sistema operativo Linux, usando Python con la biblioteca de funciones MPI. El alumno empleará un único PC para aprender los mecanismos de desarrollo y ejecución de programas paralelos que se emplean en plataformas distribuidas. En ese caso, la ejecución en un PC de la aplicación con distinto número de procesos podrá beneficiarse de los múltiples núcleos del procesador. Posteriormente, en el último ejercicio del boletín, el alumno lanzará la ejecución de su aplicación paralela en varios PC trabajando como si fueran distintos nodos de un clúster y analizará su rendimiento.

ESTUDIO PREVIO

Lea atentamente la presentación de diapositivas titulada "Introducción a la programación de computadores de memoria distribuida usando MPI" que forma parte del material de esta práctica.

REALIZACIÓN DE LA PRÁCTICA

Tras iniciar sesión en linux, abra una consola de terminal y cree un directorio de trabajo para esta práctica llamado `practica-mpi`. Cree o edite los archivos de código fuente empleando cualquier editor de texto de Linux (por ejemplo “gedit” o “nano”). El comando para ejecutar programas que usan MPI es

```
$ mpirun -np X python3 nombre_archivo.py
```

donde X es un número que indica el número de procesos que debe lanzar la aplicación. Lanzar más procesos que núcleos físicos es muy ineficiente; por lo tanto, el número de procesos no debería ser mayor que el número de núcleos físicos. Por otra parte, en los PC del aula de laboratorio el procesador dispone de núcleos de alto rendimiento (P-core) y núcleos de bajo consumo (E-core). Para realizar un estudio del factor de aceleración que se obtiene al aumentar el número de procesos, es conveniente excluir a los núcleos de bajo consumo. Para lanzar un único proceso por cada núcleo físico y para excluir los E-core, debemos ejecutar el siguiente comando:

```
$ mpirun --bind-to core -np X python3 nombre_archivo.py
```

donde X debe ser menor o igual que 8 en los PC de laboratorio.

Puede que tenga que instalar el soporte de MPI para Python en el PC del aula. Si es el caso, puede hacerlo desde la misma consola usando esta orden:

```
$ python3 -m pip install mpi4py
```

Sólo hay que hacerlo una vez en toda la sesión.

NOTAS:

- La versión de Python instalada en la partición de Ubuntu de los PCs del laboratorio es la 3, así que, para todo el texto del presente boletín, es posible usar la orden *python* en lugar de *python3*.
- Para realizar la práctica en casa, será necesario instalar también MPI en su ordenador.

PRIMERA PARTE

EJERCICIO 1.1

Modificar el programa “hola_mundo.py” (presentado en las transparencias incluidas en el material de esta práctica) para que en el mensaje de salida que muestra por pantalla cada proceso incluya al final el nombre del procesador en que está corriendo ese proceso. Por ejemplo, una salida típica podría ser:

```
[practica@pc-2-16 practica-mpi]$ mpirun -np 3 python3 hola_mundo_2.py
Yo soy el proceso 0 de 3, corriendo en pc-2-16
Yo soy el proceso 1 de 3, corriendo en pc-2-16
Yo soy el proceso 2 de 3, corriendo en pc-2-16
```

Pista: Utilizar la función *MPI.Get_processor_name()*.

Investigar si se puede lanzar un número de procesos que sea mayor que el número de núcleos del procesador utilizado y qué ocurre en ese caso.

EJERCICIO 1.2:

Cree un archivo de código fuente en Python conteniendo el programa “saludos.py” (presentado en las transparencias incluidas en el material de esta práctica) y ejecútelo sobre distinto número de procesadores. ¿Qué ocurre al ejecutar el programa sólo para un procesador? ¿Por qué?

EJERCICIO 1.3:

Tal como se indica en la presentación de diapositivas titulada “Introducción a la programación de computadores de memoria distribuida usando MPI”, la función *send* es bloqueante; sin embargo, en la implementación de MPI instalada en los laboratorios de prácticas, esta función sólo es bloqueante cuando se envían mensajes con un tamaño mayor que 4 KiBytes. Si el tamaño es menor, el sistema MPI almacena el mensaje en un *buffer* interno y la llamada a *send* termina sin esperar a que dicho mensaje llegue al receptor.

Suponga que organizamos los p procesos en anillo, de forma que el proceso de rango i (o proceso i) solo se comunica con los procesos i+1 e i-1. Dado un proceso i, llamaremos proceso *siguiente* al de rango i+1, y proceso *anterior* al de rango i-1. Obviamente, para cerrar el anillo, el último proceso (rango p-1) debe comunicarse con el primero (rango 0). Es decir, el proceso siguiente al de rango p-1 es el 0, y el anterior al de rango 0 es el proceso p-1.

A) Escriba un programa "saludos_en_anillo1.py" en el que cada proceso primero recibe un saludo del proceso anterior y después envía un mensaje al proceso siguiente[2]. Después de recibir el mensaje, cada proceso debe mostrarlo en pantalla. ¿Funciona el programa o se queda bloqueado? ¿Por qué?
B) Escriba otra versión llamada "saludos_en_anillo2.py", en la que primero se envía el saludo y después se recibe. ¿Funciona o se queda bloqueado? ¿Por qué?
C) ¿Qué ocurre en ambas versiones cuando se ejecuta el programa con un único proceso? ¿Por qué?

EJERCICIO 1.4:

A continuación, se muestran dos versiones de un programa que es un ejemplo de cálculo simple en paralelo que realiza el producto escalar de dos vectores. La primera de las versiones se encuentra en el archivo “prod_escalar_mpi.py”.

A) Estudie el funcionamiento del programa. Para ello, cambie inicialmente el número n de elementos de los vectores a un valor muy pequeño (por ejemplo 6). Ejecute el programa variando el número de procesos lanzados desde 1 hasta 3, y compruebe que los resultados son correctos. Finalmente, estudie el código de ambas versiones para entender cómo se realiza el cálculo.

---

[2]El cálculo del rango de los procesos *anterior* y *siguiente* se puede realizar utilizando una sentencia *if*. Sin embargo, también se puede obtener con las siguientes expresiones: dado un proceso i, el rango del proceso siguiente es $(i+1)\%p$, y el del proceso anterior, $(i-1+p)\%p$, donde ‘%’ es el resto de la división entera en Python.

B) Modifique el programa añadiéndole una medición del tiempo de ejecución, usando para ello la función *MPI.Wtime* junto con el método *barrier*.

C) Mida el tiempo de ejecución del programa al variar el número de procesos con el que es lanzado desde 1 hasta 6 procesos. Anote los tiempos obtenidos y calcule la aceleración para cada número de procesos. La aceleración de un programa paralelo se define como el cociente entre el tiempo de ejecución del programa para un sólo procesador y el tiempo de ejecución con *n* procesadores.

   La aceleración ideal para *n* procesos es *n.* ¿Se obtienen valores próximos a *n* en todos los casos? ¿Por qué?

D) Escriba una nueva versión del programa utilizando el método *gather* y compruebe que los resultados son correctos. No es necesario que realice mediciones de tiempo en este caso.

Programas "*prod_escalar_mpi.py*" / "*prod_escalar_mpi_reduccion.py*"

```
from mpi4py import MPI

def prod_escalar_serie (a, b, primer_elemento, n):
    suma = 0.0
    for i in range(primer_elemento, primer_elemento+n):
      suma += a[i] * b[i]
    return suma

# Número de elementos que tendrán los vectores
n = 2*2*2*2*3*3*5*7*11*13   # Usamos un múltiplo de cualquier valor <= 13

# Inicializa los vectores x,y con valores [0 1 2 3 4 0 1 2 3 4 0 1 2 3 4 0 1 2 ... ]
x = [i%5 for i in range(0,n)]
y = [i%5 for i in range(0,n)]

# Establezco comunicador global y recojo mi rango y tamaño de mi comunicador
comm = MPI.COMM_WORLD
mi_rango = comm.Get_rank()
p = comm.Get_size()

# Cada proceso calcula una parte del producto escalar (suponemos que n es múltiplo de p)
n_local = n // p
inicio_vector_local = mi_rango * n_local
suma_local = prod_escalar_serie (x, y, inicio_vector_local, n_local)
print (f"MI RANGO = {mi_rango} , SUMA LOCAL = {suma_local}")
```

(Elige esta versión en la que usamos comunicaciones simples punto a punto para enviar los resultados parciales. Archivo prod_escalar_mpi.py)

```
if mi_rango == 0:
    # Si soy el proceso 0, suma las contribuciones calculadas por cada proceso
    suma_total = suma_local
    for fuente in range(1,p):
        suma_total += comm.recv (source=fuente)
    # Muestra el resultado por pantalla
    print (f"PRODUCTO ESCALAR USANDO p={p} TROZOS DE LOS VECTORES X e Y = {suma_total}")
else:
    # Si no soy el proceso 0, le envio el producto escalar local que he calculado antes
    comm.send (suma_local, dest=0)
```

*(O bien esta otra versión en la que usamos **comm.reduce** para simplificar esta parte del programa. Archivo prod_escalar_mpi_reduccion.py)*

```
# Y usando una operación de reducción, se envían las sumas parciales
# al proceso raíz, especificando que todas las sumas parciales deben sumarse
# entre sí para obtener suma_total

suma_total = comm.reduce (suma_local, root=0, op=MPI.SUM)

if mi_rango == 0:
    # Muestra el resultado por pantalla
    print (f"PRODUCTO ESCALAR USANDO p={p} TROZOS DE LOS VECTORES X e Y = {suma_total}")
```

SEGUNDA PARTE

EJERCICIO 2.1:

Se desea calcular estadísticas básicas (suma, media, máximo y mínimo) de un vector de números reales de tamaño grande (N). El vector completo solo debe existir en el proceso 0, que se encargará de distribuir el trabajo entre los demás procesos. En este ejercicio se va a codificar y analizar paso a paso un programa Python y MPI que resuelve este problema, y luego se van a proponer una serie de ampliaciones.

A) El primer paso consiste en distribuir el trabajo (los datos) entre los diferentes procesos. Es habitual que sea la primera tarea que se debe realizar en un programa que se ejecute de forma distribuida. Para ello el proceso crea un vector de N números aleatorios (de tamaño, por ejemplo, 1.000.000). Por simplicidad, aquí se va a suponer que N es múltiplo del número de procesos.

A continuación, el vector se reparte equitativamente entre todos los procesos de forma que cada proceso recibe un bloque contiguo del vector. Para ello se usa la función *comm.Scatter* que sirve para repartir datos desde un proceso raíz a todos los demás procesos de un comunicador. Esta función de *Scatter* puede trabajar con una lista o un vector en Python, divide ese objeto (lista/vector) en partes iguales según el número de procesos disponibles, y finalmente envía la primera parte al proceso 0, la segunda al proceso 1, y así sucesivamente.

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N = 10_000_000

# ---------- Apartado A ----------
# ---------- Preparación de Datos ----------
if rank == 0:
    # Generamos los datos fuera del cronómetro (simulando la carga de disco)
    vector = np.random.rand(N)
    bloques = np.array_split(vector, size)
else:
    vector = None
    bloques = None # Inicializamos variable para el bloque secuencial

# Sincronizamos todos los procesos antes de empezar a medir
comm.Barrier()

# ========================================
# INICIO MEDICIÓN PARALELA
# ========================================
t_inicio_par = MPI.Wtime()

# 1. Distribuir
bloque_local = comm.scatter(bloques, root=0)
```

B) A continuación cada proceso debe procesar su bloque local y calcular la suma parcial, el máximo local, y el mínimo local.

```
# 1. Distribuir
bloque_local = comm.scatter(bloques, root=0)


# ---------- Apartado B ----------
# 2. Computar Localmente
suma_local = np.sum(bloque_local)
max_local = np.max(bloque_local)
min_local = np.min(bloque_local)
```

C) En el siguiente paso se ha de calcular la suma total, el máximo global, el mínimo global, y la media global. Esto se consigue combinando los resultados parciales obtenidos por los procesos. El resultado final debe mostrarse solo en el proceso 0. Para ello se va a usar la función *comm.reduce* y los operadores MPI.SUM, MPI.MAX, MPI.MIN. La función *reduce* de MPI tiene el propósito de recolectar datos de todos los procesos en un comunicador, combinarlos mediante una operación matemática o lógica y entregar el resultado final a un único proceso específico (típicamente el 0 o *root*). A diferencia de un *comm.gather* (que solo junta los datos en una lista o vector), *reduce* aplica una operación de "reducción" similar al “reduce” de OpenMP.

```
# ---------- Apartado C ----------
# 3. Reducir (Comunicación global)
suma_total = comm.reduce(suma_local, op=MPI.SUM, root=0)
max_total = comm.reduce(max_local, op=MPI.MAX, root=0)
min_total = comm.reduce(min_local, op=MPI.MIN, root=0)

# ---------- Apartado D ----------
# 4. Cálculo Final en Root
if rank == 0:
    media = suma_total / N

t_fin_par = MPI.Wtime()

# ========================================
# FIN MEDICIÓN PARALELA
# ========================================
```

D) El último paso es opcional, pero conveniente para evitar errores. Se calculan las estadísticas de forma secuencial y se comprueba que los resultados coinciden (dentro de una tolerancia)

```
if rank == 0:
    tiempo_paralelo = t_fin_par - t_inicio_par

    print("-" * 30)
    print(f"Resultados (N={N}):")
    print(f"Media: {media:.4f}")
    print(f"Máx:   {max_total:.4f} | Mín: {min_total:.4f}")
    print("-" * 30)

    # ========================================
    # MEDICIÓN SECUENCIAL (SOLO EN ROOT)
    # ========================================
    t_inicio_sec = MPI.Wtime()

    # NumPy está altamente optimizado en C, es un rival difícil
    s_sec = np.sum(vector)
    max_sec = np.max(vector)
    min_sec = np.min(vector)
    desv_sec = np.std(vector) # Calcula media internamente

    t_fin_sec = MPI.Wtime()
    tiempo_secuencial = t_fin_sec - t_inicio_sec

    # ========================================
    # COMPARATIVA Y SPEEDUP
    # ========================================
    aceleración = tiempo_secuencial / tiempo_paralelo

    print(f"Tiempo Paralelo ({size} procs): {tiempo_paralelo:.6f} seg")
    print(f"Tiempo Secuencial:          {tiempo_secuencial:.6f} seg")
    print("-" * 30)
    print(f"Speedup: {aceleración:.2f}x")
```

Ahora se van a proponer algunas modificaciones y extensiones del programa anterior.

1. Añada el cálculo de la desviación típica. Para ello primero debe calcular realmente la varianza. La desviación típica será la raíz cuadrada de la varianza. La fórmula de la desviación típica que se ha de implementar es esta:

$$\sigma = \sqrt{\frac{\sum x_i^2}{N} - \mu^2}$$

donde x^2_i es la suma de los cuadrados de los elementos del vector, μ es la media aritmética y *N* es el número de valores. Debe ampliar el código para que también calcule x^2_i de forma distribuida. El proceso 0, usando este resultado, deberá aplicar la fórmula que calcula la desviación típica.

2. Mida el tiempo de ejecución paralelo y compare el tiempo secuencial vs paralelo. Para ello use la primitiva *MPI.Wtime()* que devuelve el tiempo actual. Restando el resultado guardado en dos variables que llaman a esta función *Wtime* en un bloque de instrucciones, se puede obtener el tiempo de ejecución en segundos. Para asegurarnos que sincronizamos todos los procesos antes de empezar a medir el tiempo de todos los procesos, añada la siguiente instrucción *comm.Barrier()* después del primer *if* del código del apartado A, y antes de proceder al reparto de los datos del vector entre los diferentes procesos. De esa manera todos los procesos empiezan a la vez a continuación de la instrucción de *Barrier*.

¿Qué ejecución es más rápida? ¿Por qué? Analice los resultados y modifique el tamaño del vector y el número de procesos para intentar cambiar los resultados de los tiempos.

EJERCICIO 2.2:

En este ejercicio se implementa un programa en Python utilizando la librería mpi4py que realiza la multiplicación de dos matrices cuadradas de gran tamaño (A x B = C) de forma paralela. Se comprueba además el rendimiento de la solución paralela frente a una solución secuencial básica, analizando cómo varía el factor de aceleración al aumentar el número de procesos y el tamaño de las matrices.

A continuación se describe por pasos el código del programa.

A) De forma análoga a como se hizo en el ejercicio anterior, el primer paso consiste en la inicialización de los datos y su reparto entre todos los procesos. En este caso hay que crear las matrices A, B y C en el proceso 0 o maestro.

```
from mpi4py import MPI
import numpy as np
import time
import sys

# Función auxiliar para validación (ejecución secuencial pura)
def matriz_mul_secuencial(A, B):
    return np.dot(A, B)

def main():
    # 1. Inicialización del entorno MPI
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()  # Identificador del proceso actual
    size = comm.Get_size()  # Número total de procesos

    # Definimos el tamaño de la matriz (N x N)
    N = 1000

    # IMPORTANTE: Para este ejercicio básico, asumimos que N es divisible por size.
    # En un caso real, habría que usar Scatterv/Gatherv para trozos desiguales.
    if N % size != 0:
        if rank == 0:
            print(f"Error: N ({N}) debe ser divisible por el número de procesos ({size}).")
        sys.exit(0)

    # Calculamos cuántas filas manejará cada proceso (Chunk Size)
    filas_locales = N // size

    # ---------------------------------------------------------------------------
    # PARTE 1: Inicialización de Datos (Solo en el proceso Maestro)
    # ---------------------------------------------------------------------------
    if rank == 0:
        print(f"--- Iniciando multiplicación {N}x{N} con {size} procesos ---")
        A = np.random.rand(N, N).astype('d')
        B = np.random.rand(N, N).astype('d')
        # Matriz vacía para recibir el resultado final
        C_final = np.empty((N, N), dtype='d')

        start_time = time.time()
    else:
        # Los esclavos no necesitan tener la Matriz A completa, solo su trozo.
        # Pero sí necesitan espacio para recibir la Matriz B completa.
        A = None
        B = np.empty((N, N), dtype='d')
        C_final = None
```

B) En los dos pasos siguientes se produce la comunicación de datos entre los procesos y el cómputo de los resultados. Hay que fijarse que en el caso de la multiplicación de matrices, si bien la matriz A hay que dividirla entre todos los procesos, la matriz B ha de ser enviada completa a todos ellos porque se necesita completa en cada proceso para calcular el trozo correspondiente de la matriz C.

```
# ----------------------------------------------------------------------
# PARTE 2: Distribución de Datos (Comunicación)
# ----------------------------------------------------------------------

# 2.1 Broadcast de la matriz B
# Enviamos la matriz B completa desde root=0 a todos los procesos.
# Es crucial que B esté creada en los esclavos antes de llamar a Bcast.
comm.Bcast(B, root=0)

# 2.2 Scatter de A
# Preparamos el buffer local para recibir el trozo de filas de la matriz A.
# Dimensiones: (filas_locales x N)
A_local = np.empty((filas_locales, N), dtype='d')

# Repartimos la matriz A. MPI toma la matriz A del root, lo divide en trozos iguales y
# envía cada trozo al buffer A_local de cada proceso.
comm.Scatter(A, A_local, root=0)

# ----------------------------------------------------------------------
# PARTE 3: Cómputo Paralelo
# ----------------------------------------------------------------------

# Sincronización opcional para asegurar que todos empiezan el cálculo a la vez
comm.Barrier()

# Realizamos la multiplicación local.
# (filas_locales x N) dot (N x N) Resulta en -> (filas_locales x N)
# NumPy es muy rápido por núcleo.
# t_inicio_calculo = time.time()
C_local = np.dot(A_local, B)
# t_fin_calculo = time.time()

# (Opcional) Imprimir tiempo de cálculo por proceso para ver balanceo de carga
# print(f"Proceso {rank} terminó cálculo en {t_fin_calculo - t_inicio_calculo:.4f}s")
```

C) En el último paso se recogen los resultados parciales de la matriz C y se hacen una serie de comprobaciones en el proceso 0 o maestro para ver si realmente el resultado obtenido en la matriz C usando MPI es equivalente al obtenido de forma secuencial. También en el proceso 0 o maestro se calculan los tiempos de ejecución tanto de forma paralela como secuencial y se obtiene el factor de aceleración.

```
    # ----------------------------------------------------------------------
    # PARTE 5: Recolección (Modificado)
    # ----------------------------------------------------------------------
    # Ahora reunimos la matriz A
    # Reunimos los trozos C_local en C_final en el root.
    # MPI toma C_local de cada proceso y los pega en orden en C_final.
    comm.Gather(C_local, C_final, root=0)


    # ----------------------------------------------------------------------
    # PARTE 6: Validación y Resultados (Solo Maestro)
    # ----------------------------------------------------------------------
    if rank == 0:
        end_time = time.time()
        tiempo_total = end_time - start_time
        print(f"Tiempo Total Paralelo: {tiempo_total:.4f} segundos")

        # Validación: Solo si la matriz no es gigantesca (por tiempo)
        if N <= 1000:
            print("Validando resultado con cálculo secuencial...")
            start_seq = time.time()
             # 1. Cálculo secuencial base
            C_check = matriz_mul_secuencial(A, B)

            # 2. Normalizar el resultado secuencial. Descomentar cuando se haya resuelto la ampliación propuesta.
            # min_check = np.min(C_check)
            # max_check = np.max(C_check)
            # C_check = (C_check - min_check) / (max_check - min_check)

            end_seq = time.time()
            print(f"Tiempo Secuencial (un solo núcleo): {end_seq - start_seq:.4f} segundos")

            # np.allclose es mejor que == por errores de redondeo en flotantes
            if np.allclose(C_final, C_check):
                print(">> ÉXITO: El resultado paralelo es correcto.")
                print(f">> Speedup aproximado: {(end_seq - start_seq)/tiempo_total:.2f}x")
            else:
                print(">> ERROR: Los resultados no coinciden.")
        else:
            print("Validación omitida por tamaño de matriz.")

if __name__ == "__main__":
    main()
```

El objetivo del ejercicio es añadir a este código una “PARTE 4” de post-procesamiento distribuido. Esto simula nuevamente un flujo de trabajo real típico en aplicaciones de *Big Data*: Calcular -> Analizar/Transformar -> Reunir.

¿En qué consiste esta ampliación del código?

En lugar de simplemente multiplicar y devolver el resultado, se deben tomar la matriz resultante C (que está distribuida en trozos en la memoria de cada proceso) y realizar una *Normalización Min-Max* antes de reunir los datos.

La normalización Min-Max es una técnica que transforma los valores de la matriz original C a un rango estándar [0, 1], lo cual es una operación común en el preprocesamiento de datos para *Machine Learning*. La fórmula que cada proceso debe aplicar a sus elementos locales es:

$$C_{i,j}^{norm} = \frac{C_{i,j} - \min(C_{global})}{\max(C_{global}) - \min(C_{global})}$$

Donde:

- $C_{i,j}$ es el valor original del elemento en la fila *i* y columna *j*.

-min(C_{global}) y max(C_{global}) son los valores mínimo y máximo de toda la matriz resultante, no solo del trozo local de cada proceso.

- $C_{i,j}^{norm}$ es el valor normalizado resultante en el rango [0, 1].

Cada proceso tiene una parte de la matriz y por ello solo conoce el mínimo y máximo de su trozo. Pero para normalizar necesitan saber el mínimo y máximo global (de toda la matriz). La matriz C está distribuida entre varios procesos, y ningún proceso conoce por sí solo los valores min(C_{global}) y max(C_{global}).

Para ello hay que entender que no sirve usar *Reduce* (que envía el resultado solo al maestro), sino que se necesita usar *Allreduce* (que envía el resultado a todos), o una combinación de *Reduce* + *Bcast*. Esta es la “PARTE 4” que hay que añadir y completar al código original. En la “PARTE 6” hay también que descomentar las líneas del paso 2.

```
# ---------------------------------------------------------------------------
# PARTE 4: Recolección de Resultados (Comunicación) (RELLENAR ALUMNOS)
# ---------------------------------------------------------------------------
# Objetivo: Normalizar la matriz resultado C para que todos sus valores
# estén entre 0 y 1.
# Fórmula: C_norm = (C - Min_Global) / (Max_Global - Min_Global)

# RETO:
# 1. C_local solo tiene una parte de los datos.
# 2. Tenéis que encontrar el Mínimo y Máximo de TODA la matriz C,
#    sin reunir todavía la matriz en el maestro (sería ineficiente).
# 3. Una vez tengáis Min_Global y Max_Global en TODOS los procesos,
#    aplicad la fórmula a vuestro C_local.

# PISTA: Buscad en la documentación de MPI las operaciones de reducción
# (Reduce, Allreduce) y los operadores (MPI.MIN, MPI.MAX).

# --- ESCRIBIR CÓDIGO AQUÍ ---
```

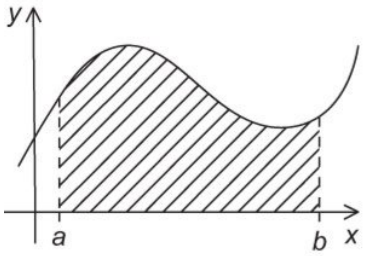
En este ejercicio se utiliza la función dot() de Numpy, que realiza la multiplicación de matrices. Esta función se ejecuta en múltiples hilos (está programada con OpenMP). Esto puede distorsionar el análisis del rendimiento frente al número de procesos, ya que cuando lancemos, por ejemplo, dos procesos en 8 núcleos, la función dot() va a intentar usar todos los núcleos para realizar el cálculo. Para evitar este problema debemos indicar a Numpy que use un solo hilo. Para ello, en el término donde vayas a lanzar la ejecución con mpirun, escribe primero el siguiente comando:

```
export OMP_NUM_THREADS=1
```

EJERCICIO 2.3:

A continuación, se presenta un programa en *Python* que realiza una estimación numérica de la integral o área encerrada entre la gráfica de una función no negativa f(x), dos líneas verticales situadas en posiciones x=a y x=b y el eje x, utilizando el "método de los trapecios".

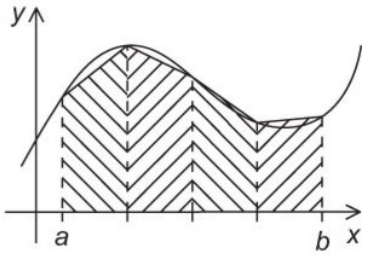
$$S = \int_a^b f(x)\,dx$$



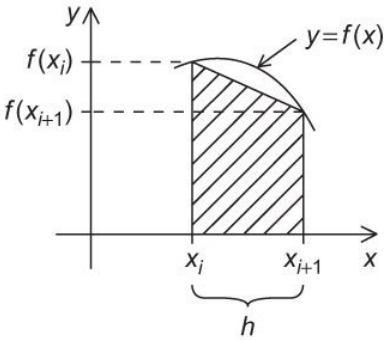
Programa "integracion_trapecios_no_paralelo.py"

```
# INTEGRACIÓN NUMÉRICA POR EL MÉTODO DE LOS TRAPECIOS
# ENTRADA: NINGUNA.
# SALIDA: ESTIMACIÓN DE LA INTEGRAL DESDE a HASTA b DE f(x)
# USANDO EL MÉTODO DE LOS TRAPECIOS CON n TRAPECIOS

# FUNCIÓN QUE QUEREMOS INTEGRAR
def f(x):
    return (1-x*x)**0.5

a = -1.0        # EXTREMO IZQUIERDO
b = 1.0         # EXTREMO DERECHO
n = 100000      # NUMERO DE TRAPECIOS
h = (b-a)/n     # LONGITUD DE LA BASE DEL TRAPECIO

# COMIENZA EL CÁLCULO DE LA INTEGRAL

integral = 0
for i in range(1,n):
    integral += f(a+i*h)

integral += (f(a) + f(b))/2.0
integral *= h

print (f"ESTIMACIÓN USANDO n={n} TRAPECIOS")
print (f"DE LA INTEGRAL DESDE {a} HASTA {b} = {integral}")
print (f"ESTIMACIÓN DE PI: {2*integral}")
```

El método de los trapecios consiste en subdividir el intervalo sobre el eje x en *n* subintervalos idénticos y aproximar el área encerrada entre la curva y cada subintervalo por un trapecio cuya base es el subintervalo, tal como se muestra en la siguiente figura.



El área del subintervalo (o trapecio) i-ésimo se puede calcular fácilmente como la suma de las áreas de un rectángulo y un triángulo:



$$S\left(\text{trapecio}_i\right) = h \cdot f\left(x_{i+1}\right) + \frac{1}{2}h \cdot \left[f\left(x_i\right) - f\left(x_{i+1}\right)\right] = \frac{1}{2}h \cdot \left[f\left(x_i\right) + f\left(x_{i+1}\right)\right]$$

El área total puede aproximarse sumando el área de los *n* trapecios, definidos por los pares de puntos $\left(x_0,x_1\right), \left(x_1,x_2\right), \ldots, \left(x_{n-1},x_n\right)$. Dicha suma es igual a

$$S \approx \sum_{i=0}^{n} S\left(\text{trapecio}_\text{i}\right) = \left[\frac{f\left(x_0\right)}{2} + \frac{f\left(x_n\right)}{2} + f\left(x_1\right) + f\left(x_2\right) + \cdots + f\left(x_{n-1}\right)\right] h$$

donde x_0 = a y x_n = b. Esta es la fórmula de los trapecios usada en el programa anterior considerando que la base de todos los trapecios es de la misma longitud: h = (b-a) / n.

A partir de este programa, **debe realizar una versión paralela del mismo utilizando MPI**. En este caso, solo se necesita paralelizar el cómputo realizado por el bucle *for*. Por lo tanto, deben repartirse entre los diferentes procesos los cálculos realizados en los *n*-1 puntos $x_1, \ldots, x_{n-1}$; es decir, debemos repartir las *n*-1 iteraciones del bucle *for* entre los procesos. Esto es posible porque se trata de un bucle sin dependencia de datos. A continuación, proponemos tres posibles formas de repartir las iteraciones:

- Sea *p* el número de procesos. Si suponemos que *n*-1 es múltiplo de *p*, entonces el número de iteraciones que debe ejecutar cada proceso es `numiter=(n-1)//p` (división entera de Python). El proceso 0 ejecutará las primeras *numiter* iteraciones (*i* = 1*, ..., numiter*); el proceso 1, las siguientes *numiter* iteraciones (*i* = 1+*numiter*, ..., 2×*numiter*), etc. Es decir, cada proceso ejecutará las iteraciones desde *i* igual a
  ```
  1 + rango * numiter
  ```
  hasta
  ```
  (rango+1) * numiter
  ```
  ambas incluidas.
- Para repartir las iteraciones sin necesidad de suponer que *n*-1 sea múltiplo de *p*, se puede calcular el valor inicial de *i* de cada proceso con la expresión
  ```
  1 + (rango * (n-1)) // p
  ```
  y el valor final, con
  ```
  ((rango+1) * (n-1)) // p
  ```
  Al tratarse de divisiones enteras, el producto debe realizarse antes de la división (se han usado paréntesis redundantes para remarcar este hecho).
- Otra forma de repartir las iteraciones entre los p procesos sería la siguiente:

El proceso 0 ejecuta las iteraciones para i = 1, 1+p, 1+2p, 1+3p, ...
El proceso 1 ejecuta las iteraciones para i = 2, 2+p, 2+2p, 2+3p, ...
El proceso 2 ejecuta las iteraciones para i = 3, 3+p, 3+2p, 3+3p, ...
...

Es decir, para cada proceso, el valor de i se inicializa con 1 + *rango* y se va incrementando en *p* unidades. Debido a que este mecanismo es menos general que los anteriores, recomendamos no utilizarlo en este ejercicio.

El algoritmo paralelo quedaría, por tanto, así:

1. Cada proceso calcula qué iteraciones debe ejecutar a partir de su rango.
2. Cada proceso ejecuta las iteraciones que le corresponden y calcula la suma parcial.
3. Cada proceso distinto del 0 envía su cálculo al proceso 0 (usando la función *send*).
4. El proceso 0 recibe los cálculos del resto de procesos (usando la función *recv*), los suma a su propio cálculo, termina de calcular la integral y muestra el resultado final.

Para adaptar el programa secuencial a su versión paralela, se recomienda organizar el programa de forma similar a como se hizo con el cálculo del producto escalar en el ejercicio 1.4. Es decir, añada una función que reciba como argumentos, por ejemplo, los valores de *a*, *n*, *h*, *p* y *rango*. Dicha función calcula la suma parcial (pasos 1 y 2 de la enumeración anterior) que le corresponde al proceso y la devuelve como resultado.

Para probar el programa obtendremos una estimación del valor de π a partir del área de medio círculo. Para conseguirlo, debe calcularse el área de medio círculo de radio unidad. Puesto que la ecuación de un círculo centrado en el origen es $x^2 + y^2 = r^2$, debemos integrar la función $y = \sqrt{r^2 - x^2}$ para obtener el área[3], que es igual a π/2 (ya que el área de un círculo de radio unidad es π). Por lo tanto, multiplicando por 2 el valor calculado por nuestro programa obtendremos una estimación de π. Si en su solución ha optado por suponer que *n*-1 es múltiplo del número de procesos, entonces pruébelo con 1 y con 3 procesos.

EJERCICIO 2.4:

Realice los siguientes cambios en el programa anterior:

- Modifique el programa para que calcule la integral de la función logaritmo log(x) (logaritmo natural de x, que se encuentra disponible en el paquete *math* de Python). Inicialice las variables con los siguientes valores:
  - Extremo izquierdo: a = 1.0
  - Extremo derecho: b = 100000.0
  - Número de trapecios: n = 7927920, que es el resultado de 2*2*2*2*3*3*5*7*11*11*13

  El resultado del área debe ser aproximadamente 1051293.5464837584.

- Añada al programa una medida del tiempo de ejecución, usando la función *MPI.Wtime* junto con *barrier*.

Ejecute el programa y anote el tiempo de ejecución obtenido para un número de procesos desde 1 hasta 7. Represente una gráfica del tiempo de ejecución respecto al número de procesos.

Represente en una gráfica la aceleración respecto al número de procesos. La aceleración de un programa paralelo se define como el cociente entre el tiempo de ejecución del programa para un sólo procesador y el tiempo de ejecución con n procesadores. Con los datos anteriores, calcular la aceleración del programa para n desde 2 hasta 7 procesos.

Interprete ambas gráficas.

EJERCICIO 2.5:

A) Escriba una nueva versión del programa utilizando el método *gather* y compruebe que los resultados son correctos. No es necesario que realice medidas de tiempo en este caso.
B) Escriba una nueva versión del programa utilizando el método *reduce* y compruebe que los resultados son correctos. No es necesario que realice medidas de tiempo en este caso.

---

[3] Para calcular la raíz cuadrada en Python, puede elevar la expresión al valor 0.5 con el operador **, o utilizar la función *sqrt* disponible en el paquete *math*.

EJERCICIO 2.6:

Trabajando en grupos de 2 alumnos, **lanzar la aplicación paralela desarrollada en el ejercicio 2.2 en tres PC del aula. Los alumnos deberán configurar los ordenadores para ello**, teniendo en cuenta las indicaciones que se ofrecen a continuación e investigando en Internet para buscar información adicional.

Para que MPI sea capaz de lanzar procesos en otras máquinas, la cuenta debe estar configurada de modo que MPI se pueda conectar remotamente en ellas, sin que se le pida contraseña, mediante una conexión segura usando el protocolo de conexión remota con criptografía SSH (Secure Shell). Para ello hay que habilitar el acceso remoto sin contraseña desde el nodo que lanza la aplicación paralela (lo llamaremos nodo consola, o consola) a todos los nodos adicionales sobre los que deba ejecutarse. Por ejemplo, si se quiere lanzar la aplicación paralela desde el nodo pc-14-145, corriendo además de en éste en los nodos pc-14-146 y pc-14-147, el procedimiento es:

1. Activar el demonio SSH en todos los nodos adicionales donde se vaya a correr el programa. Haced doble clic en el icono señalizado a tal efecto en el escritorio de Ubuntu.
2. Generar la pareja de claves SSH pública y privada del equipo que lanza el programa (tipo RSA). Para ello, desde el nodo consola, pc-14-145 en nuestro ejemplo, ejecutar

```
$ ssh-keygen
```

3. Copiar e instalar la clave pública en todos los ordenadores adicionales donde se vaya a correr el programa. Desde el nodo consola hacer:

```
$ ssh-copy-id practica@pc-14-146
$ ssh-copy-id practica@pc-14-147
```

4. Desde el nodo consola, copiar el directorio completo de trabajo donde esté incluido el script Python a todos los ordenadores adicionales donde se vaya a correr el programa. Por ejemplo, si dicho directorio es /home/practica/mpi-001, habrá que situarse en /home/practica y ejecutar:

```
$ scp -r ./mpi-001 practica@pc-14-146:/home/practica
$ scp -r ./mpi-001 practica@pc-14-147:/home/practica
```

5. Ejecutar la aplicación paralela indicando explícitamente el nombre (o dirección IP) de todos los nodos sobre los que se vaya a ejecutar en paralelo. Por ejemplo, para lanzar un total de 8 procesos que se ejecutarán en los nodos pc-14-145, pc-14-146 y pc-14-147, debe escribir el siguiente comando:

```
$ mpirun -np 8 –mca btl_tcp_if_include enp2s0 --oversubscribe -host pc-14-145,pc-14-
146,pc-14-147 python /home/practica/mpi-001/ej4-integracion_trapecios_timer.py
```

enp2s0 es el nombre de la interfaz de red principal del equipo, conectada a la red local del laboratorio. En vuestra aula/PC el nombre podría ser otro. Comprobarlo usando la orden:

```
$ ip addr
```

Y buscar el nombre de la interfaz asociada a la red 10.0.x.x

Previo a la ejecución del ejercicio 2.2 en esta nueva configuración, se aconseja primero probar con el ejercicio 1.2 para comprobar que realmente se están ejecutando copias del script en nodos remotos. A ese ejercicio, añadir en el mensaje el nombre del nodo desde el que se envía el mensaje como se pidió hacer para el ejercicio 1.1. La razón de usar este ejercicio 1.2 modificado en lugar del 1.1 es que ahora, con la ejecución posible en nodos que no son la consola, no podemos asegurar que se pueda ver en la consola la salida de print proveniente de esos nodos. El programa saludos.py del ejercicio 1.2 sólo usa salida por consola en el proceso 0, que típicamente es el que se lanza en la consola.

Con los mismos datos de ejecución indicados para el apartado 2, ejecutar el programa y anotar el tiempo de ejecución obtenido para un número de procesos desde 1 hasta 12. **Representar una gráfica del tiempo de ejecución y otra de la aceleración respecto al número de procesos.**

Interpretar ambas gráficas, comparándolas con las obtenidas en el ejercicio 2.2. Razonar a qué se deben las diferencias encontradas. ¿Se ha conseguido sacar rendimiento a la posibilidad de conectar entre sí varios PC, tal como en un clúster, para ejecutar aplicaciones paralelas?